



Tanstack Query

In React is er **geen ingebouwde standaard voor data fetching**. Elke ontwikkelaar doet het net even anders.



```
import { useState, useEffect } from 'react'

export function useUsers() {
  const [users, setUsers] = useState([])
  const [isLoading, setIsLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    const fetchUsers = async () => {
      try {
        const res = await fetch('/api/users')
        if (!res.ok) throw new Error('Fout bij ophalen')
        const data = await res.json()
        setUsers(data)
      } catch (err) {
        setError(err)
      } finally {
        setIsLoading(false)
      }
    }

    fetchUsers()
  }, [])

  return { users, isLoading, error }
}
```

Data Fetching

Fetching data from a server or other data source is a key part of most applications. Doing this properly requires handling loading states, error states, and caching the fetched data, which can be complex.

Purpose-built data fetching libraries do the hard work of fetching and caching the data for you, letting you focus on what data your app needs and how to display it. These libraries are typically used directly in your components, but can also be integrated into routing loaders for faster pre-fetching and better performance, and in server rendering as well.

Note that fetching data directly in components can lead to slower loading times due to network request waterfalls, so we recommend prefetching data in router loaders or on the server as much as possible! This allows a page's data to be fetched all at once as the page is being displayed.

If you're fetching data from most backends or REST-style APIs, we suggest using:

- [React Query](#)
- [SWR](#)
- [RTK Query](#)

If you're fetching data from a GraphQL API, we suggest using:

- [Apollo](#)
- [Relay](#)

Verschillende soorten **state**

Voordat we TanStack Query bekijken, moeten we één ding duidelijk krijgen:

Er bestaan twee fundamenteel verschillende soorten state in een frontend app.

En ze vragen om verschillend gereedschap.

Client state VS Server State

- Is de modal open?
- Wat staat er in het formulier?
- Welke tab is actief?
- Dark mode aan of uit?
- Lijst van todos
- User profile
- Transacties
- Producten, posts, comments

Server state eigenschappen

- Het kan verouderen zonder dat je het weet
- Het wordt gedeeld tussen componenten
- Het moet soms opnieuw opgehaald worden
- Identieke gelijktijdige requests moeten samengevoegd worden

Wat is **tanstack query**

TanStack Query is een tool voor het slim beheren van server data in je frontend app.

Het regelt automatisch het ophalen, cachen, updaten en synchroniseren van API-data — zonder gedoe met boilerplate code.

Powerful tanstack **hooks**

- **useQuery** – om data op te halen van een server (GET).
- **useMutation** – voor het aanpassen van data (POST, PUT, DELETE).
- **QueryClient** – beheert de cache en query-logica centraal.

Query Keys

Query Keys zijn **unieke identificaties** voor elke query in TanStack Query.

Ze vertellen TanStack Query welke data het precies moet ophalen, cachen of bijwerken.



```
const userId = 'b42fe530-9ce6-4824-97b2-8593db2a5610';
```

```
const { data, isLoading } = useQuery(  
  {  
    queryKey: ['user', userId],  
    queryFn: () => fetchUserById(userId)  
  });
```

Caching

Caching betekent dat eerder opgehaalde data tijdelijk wordt opgeslagen (bijv. in geheugen), zodat je die niet opnieuw van de server hoeft te halen.

Dit versnelt de gebruikerservaring én vermindert het aantal netwerkverzoeken.



```
const query1 = useQuery({  
  queryKey: ['todos'],  
  queryFn: fetchTodos,  
  staleTime: 5 * 60 * 1000,  
});
```

```
const query2 = useQuery({  
  queryKey: ['todos'],  
  queryFn: fetchTodos,  
  staleTime: 5 * 60 * 1000,  
});
```



```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: Infinity,
      gcTime: 5 * 60 * 1000
    },
  },
})

function App() {
  return (
    // Provide the client to your App
    <QueryClientProvider client={queryClient}>
      <Todos />
    </QueryClientProvider>
  )
}
```


staletime & gcTime

- **staletime** bepaalt hoelang de data als “vers” gezien moet worden (default is: 0)
- **gcTime** (vroeger cacheTime) bepaald hoe lang de data in memory bewaard moet worden (default is 5min). gcTime telt pas af als er geen component meer actief gebruik maakt van de query



```
export const useCurrentUser = () =>
  useQuery({
    queryKey: ['user', 'me'],
    queryFn: userService.getMe,
    staleTime: 30 * 1000, // 30 seconden
    gcTime: 5 * 60 * 1000, // 5 minuten
  });
```



- 🕒 00:00 Header mount
→ Cache leeg → HTTP call → data binnen
- 🕒 00:15 Rerender (parent state verandert)
→ Niks. Rerender ≠ event.
- 🕒 00:45 Rerender (data is nu stale)
→ Nog steeds niks. Rerender ≠ event.
- 🕒 01:00 Header unmount (andere layout)
→ gcTime timer start
- 🕒 01:30 Header mount opnieuw
→ Cache hit (instant UI)
→ Stale → background refetch
→ Verse data komt stil binnen
- 🕒 10:00 Je zit al lang op een andere page
→ gcTime voorbij → data uit cache gegooit

Wat is de `queryClient`

- de cache van queries
- het invalidatie- en refetch-mechanisme
- achtergrond updates
- mutaties
- garbage collection van oude data



```
import { useMutation, useQueryClient } from '@tanstack/react-query'

const queryClient = useQueryClient()

const mutation = useMutation({
  mutationFn: addTodo,
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['todos'] })
    queryClient.invalidateQueries({ queryKey: ['reminders'] })
  },
})
```



```
export const useTransactions = (filters: TransactionFilters) =>
  useQuery({
    queryKey: TRANSACTIONS_QUERY_KEYS.transactions(filters),
    queryFn: ({ queryKey }) =>
      transactionsService.getTransactions(
        queryKey[1] as TransactionFilters,
      ),
  });
```



```
export const TRANSACTIONS_QUERY_KEYS = {  
  transactions: (filters: TransactionFilters) => ['transactions', filters],  
  transaction: (id: string) => ['transactions', id],  
};
```



```
const { data: transactions, isPending, isError
} = useTransactions({
  location: location?.name,
  inTransit: false,
  size: 5,
});
```


